

1. LL(1) é mais intuitivo e fácil de implementar manualmente, pois segue a estrutura lógica do código da esquerda para a direita e ainda facilita o tratamento de erros e a inserção de ações semânticas durante a análise. Porém ela é menos poderosa do que o LALR, já que aceita uma classe menor de gramáticas, e não suporta recursividade a esquerda. ANTLR e JavaCC usam métodos top-down e YACC e Bison usam métodos bottom-up.

2. O escopo de uma declaração de x é a região do programa em que os usos de x se referem a essa declaração. Uma linguagem usa escopo estático ou escopo léxico se for possível determinar o escopo de uma declaração examinando-se apenas o programa. Caso contrário, a linguagem utiliza escopo dinâmico. Com o escopo dinâmico, enquanto o programa é executado, o mesmo uso de x poderia referir-se a qualquer uma dentre as várias declarações diferentes de x . C e Java utilizam escopo estático, enquanto bash e Lisp utilizam escopo dinâmico.

3. - Área de código:

- Área onde são armazenados as instruções executáveis do programa;
- Contém o código binário compilado das suas funções.

- Dados estáticos:

- Área de memória reservada no início da execução do programa e liberada no fim da execução;
- Contém variáveis globais e variáveis declaradas como static.

- Heap:

- Área reservada para alocação dinâmica. Permite alocar e liberar espaços de memória explicitamente quando necessário.
- Contém blocos de memória alocados com malloc, que permanecem lá até serem liberados pela função free.

- Pilha:

- Área de armazenamento temporário utilizada para gerenciar a execução de funções. Ela cresce e diminui conforme as funções são chamadas e retornam.
- Contém variáveis locais declaradas dentro de funções (sem static), argumentos passados para a função e endereço de retorno.

4. - Escopo global: em C são as variáveis declaradas fora de qualquer função. Podem ser acessadas por qualquer parte do programa após a sua declaração.

- Escopo local: são variáveis declaradas dentro de uma função ou um bloco de código. São visíveis apenas dentro da função ou bloco em que foram declaradas.

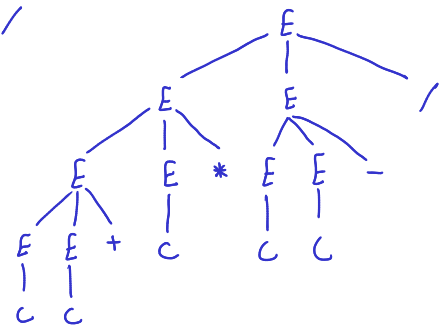
5. - Política de Análise Estática: a análise estática ocorre em tempo de compilação, antes de o programa ser executado. O compilador examina o código-fonte para garantir que ele obedeça às regras sintáticas e semânticas da linguagem.

- Política de Análise Dinâmica: a análise dinâmica ocorre em tempo de execução, enquanto o programa está rodando. O ambiente de execução verifica comportamentos que dependem dos valores dos dados ou do fluxo de controle real.

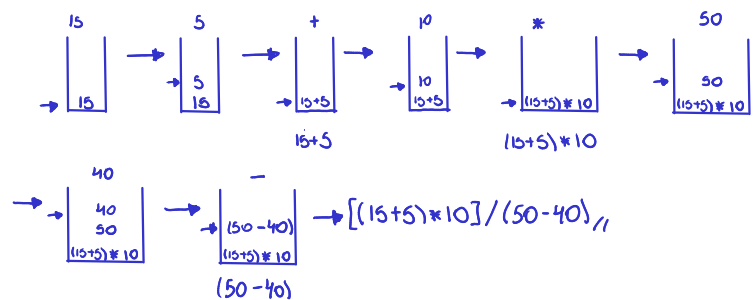
6. A diferença fundamental está no fato de que na passagem por valor a função recebe apenas uma cópia do dado, impedindo que alterações no parâmetro afetem a variável original, enquanto na passagem por referência é enviado o endereço de memória, permitindo que a função modifique diretamente a variável externa. Na linguagem Java, acontece estritamente a passagem por valor em todas as situações; para tipos primitivos, o valor real é copiado para a pilha de execução, e para objetos, o que é copiado é o valor de referência (o endereço de memória), o que permite alterar os atributos internos do objeto apontado, mas impede que a variável original seja reatribuída para um novo objeto.

7. $E \rightarrow EE +$
 $| EE -$
 $| EE *$
 $| EE /$
 $| c$

$cc + c * cc - /$
 $E \rightarrow EE / \rightarrow EE * EE - / \rightarrow EE + c * cc - /$
 $\rightarrow cc + c * cc - /$



8. $15 \ 5 \ + \ 10 \ * \ 50 \ 40 \ - \ /$



9.	1. $P \rightarrow T \text{ id } (L)$	Sequintes	$I_0: [P' \rightarrow P \#]_1$
	2. $T \rightarrow \text{int}$	$P - \{ \# \}$	$[P \rightarrow T \text{ id } (L)]_2$
	3. $T \rightarrow \text{float}$	$T - \{ \text{id} \}$	$[T \rightarrow \text{int}]_3$
	4. $L \rightarrow L, T \text{ id}$	$L - \{ \}, \{ \}, \{ \}$	$[T \rightarrow \text{float}]_4$
	5. $L \rightarrow T \text{ id}$		

$I_1: \text{Desvio}(0, P)$ $[P' \rightarrow P. \#]_{AC}$	$I_2: \text{Desvio}(0, T)$ $[P \rightarrow T. \text{ id } (L)]_5$	$I_3: \text{Desvio}(0, \text{int})$ $[T \rightarrow \text{int.}]_{R_2 = \text{Sequintes}(T)}$
---	--	--

$I_4: \text{Desvio}(0, \text{float})$ $[T \rightarrow \text{float.}]_{R_3 = \text{Sequinte}(T)}$	$I_5: \text{Desvio}(2, \text{id})$ $[P \rightarrow T \text{ id. } (L)]_6$	$I_6: \text{Desvio}(5, L)$ $[P \rightarrow T \text{ id } (L)]_7$ $[L \rightarrow L, T \text{ id}]_7$ $[L \rightarrow T \text{ id}]_8$ $[T \rightarrow \text{int}]_3$ $[T \rightarrow \text{float}]_4$
---	--	--

$I_7: \text{Desvio}(6, L)$ $[P \rightarrow T \text{ id } (L)]_9$ $[L \rightarrow L, T \text{ id}]_{10}$	$I_8: \text{Desvio}(6, T)$ $[L \rightarrow T. \text{ id}]_{11}$	$I_9: \text{Desvio}(7,)$ $[P \rightarrow T \text{ id } (L)]_{R_1 = S(P)}$
---	--	---

$I_{10}: \text{Desvio}(6,)$ $[L \rightarrow L, T \text{ id}]_{12}$ $[T \rightarrow \text{int}]_3$ $[T \rightarrow \text{float}]_4$	$I_{11}: \text{Desvio}(8, \text{id})$ $[L \rightarrow T \text{ id.}]_{R_5 = S(L)}$	$I_{12}: \text{Desvio}(10, T)$ $[L \rightarrow L, T. \text{ id}]_{13}$
--	---	---

$I_{13}: \text{Desvio}(12, \text{id})$
 $[L \rightarrow L, T \text{ id.}]_{R_4 = S(L)}$

	id	(	)	int	float	,	#	P	T	L
0				3	4			1	2	
1							AC			
2	5									
3	R_2									
4	R_3									
5		6								
6				3	4				8	7
7			9			10				
8	11									
9							R_1			
10				3	4				12	
11			R_5				R_5			
12	13									
13			R_4				R_4			

11. Serve como um repositório central de dados onde o compilador armazena informações essenciais sobre os identificadores encontrados no código-fonte, como nomes de variáveis, funções e constantes, associando-os aos seus respectivos atributos, como tipo de dado, escopo e endereço de memória. Essa estrutura é fundamental para garantir a consistência semântica do programa, permitindo que o compilador verifique se uma variável foi declarada antes do uso e se os tipos operados são compatíveis, além de auxiliar na geração do código final ao mapear onde cada dado deve ser alocado.

12. A AST é uma representação hierárquica e simplificada da estrutura do código-fonte que elimina os detalhes puramente sintáticos da gramática, como parênteses, ponto e vírgula e palavras-chave delimitadoras, mantendo apenas as informações essenciais sobre as operações e seus operandos. Ela organiza a lógica do programa de forma condensada, onde os operadores geralmente aparecem como nós internos e os identificadores ou constantes como folhas, servindo como a estrutura de dados principal para realizar a análise semântica e facilitar a geração de código intermediário ou de máquina.

13. - Erro léxico:

- $i = 1A;$
- O analisador léxico falha ao tentar reconhecer o token 1A. Em C, um identificador não pode começar com dígito, e uma constante numérica não pode ser seguida imediatamente por uma letra, exceto sufixos válidos como L, f, ou hexadecimais. Porém 1A não se encaixa em nenhum desses.

- Erro Sintático:

- while ($i \leq 5$;
- Falta de parêntese de fechamento. analisador sintático espera um) antes do {, mas encontra { inesperadamente.

- Erro Semântico:

- $v[i] = i + 10;$
- A variável i foi declarada como float, em C índices de vetores devem ser do tipo inteiro. O analisador semântico detecta que o tipo usado para indexar é incompatível com a operação de acesso ao array.

- 14.

iconst_2 ; carrega constante 2
istore_1 ; a = 2

iconst_5 ; carrega constante 5
istore_3 ; c = 5

iconst_0 ; carrega constante 0
istore_2 ; b = 0

iload_1 ; carrega a
ifle FIM ; se $a \leq 0$ pula para o fim
iload_1 ; carrega a
iload_3 ; carrega c
bipush 30 ; carrega constante 30
iadd ; soma $(c+30)$
if_icmpge FIM ; se $a \geq (c+30)$ pula para o fim

iload_1 ; carrega a
bipush 10 ; carrega constante 10
iadd ; soma $(a+10)$
bipush 2 ; carrega constante 2
imul ; multiplica $(a+10)*2$
iload_3 ; carrega c
iadd ; soma $((a+10)*2)+c$
istore_2 ; b = $((a+10)*2)+c$

FIM:

```
getstatic java/lang/System/out Ljava/io/PrintStream;
iload_2 ;carrega b
invokevirtual java/io/PrintStream/println(I)V
```